LEGACY LOGIC. MODERN ACCESS.

# Turn legacy code into reliable, agent-ready APIs and MCP servers.

Connect what runs your business
to what comes next.



01 — WHY NOW

# Enterprise AI needs the business logic it cannot reach.

Agents need more than documents. They need operational tools.

WHAT ENTERPRISES ALREADY HAVE

## Decades of business logic

Rules, records, inventory, workflows.

?

WHAT AGENTS NEED

## Reliable, callable tools

Structured access to real operations.

THE ACCESS GAP

02 — THE PROBLEM

# The logic runs.
# The systems stay siloed.

01 / LEGACY LANGUAGES

## Hard to expose

COBOL, old Java, Fortran.
Critical rules live in code.

02 / ACQUIRED STACKS

## Hard to connect

Each acquisition adds
another system and interface.

03 / INTEGRATION TEAMS

## Hard to maintain

Custom access layers demand
ongoing engineering work.

03 — THE ALTERNATIVES

# Access is only half the job.

The connection also has to be controlled, tested, and maintained.

## Custom wrappers

Hand-built interfaces and ongoing upkeep.

Engineering effort grows as source code changes.

## Direct AI access

A route to the source without a controlled layer.

Tests, traceability, and execution controls still matter.

## REPLIC8

Generate tools from source. Verify before publication.

MCP + REST, source links, and tested versioned repairs.

04 — THE SOLUTION

# Paste a repo.
# Publish tested tools.

From legacy source to agent access, through one verification pipeline.

01

## Ingest

Fetch source.
Detect the runtime.

→

02

## Extract

Find operations.
Link to file and line.

→

03

## Verify

Build in a sandbox.
Run contract tests.

→

04

## Publish

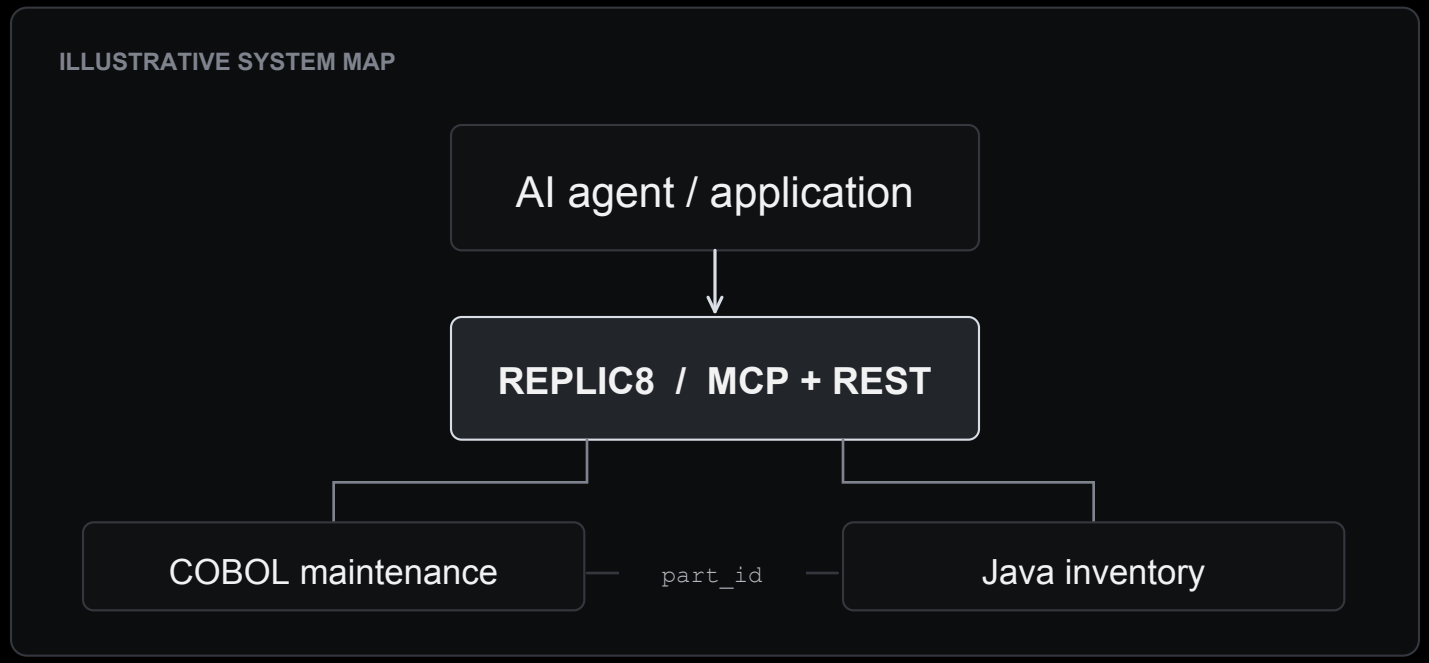
Expose MCP tools
and REST / OpenAPI.

Only operations that pass are published.

05 — THE PRODUCT

# Many systems. One gateway.

A shared interface for agents. A clear map for engineers.




## Trace every tool

Source file, line, tested commit.

## See shared fields

Map links across legacy systems.

## Operate in one place

Workspaces, tests, activity, rollback.

06 — SAFETY

# Verification comes before publication.

READ-ONLY BY DEFAULT

## The model proposes. REPLIC8 checks.

Legacy execution stays inside an offline Docker sandbox.

01 **Constrain execution**
No network. Read-only files. Resource limits.

02 **Test real behavior**
Recorded inputs and outputs gate publication.

03 **Control access**
Tokens scope which systems callers can see.

04 **Keep actions read-only**
Write operations are blocked in the MVP.

07 — RESILIENCE

# When source changes, the integration evolves.

Repair is a checked transition to a new version.

1 → 2 → 3 → 4

## Detect

Compare source
with its tested commit.

## Diagnose

Inspect the diff.
Find the changed tool.

## Replay

Repair the mapping.
Replay recorded tests.

## Version

Save only on pass.
Keep rollback available.

- The live tool stays pinned to tested code while the next version is checked.

08 — PROOF

# A COBOL rename.
# A tested repair. A new version.

## ~6s

**Recorded repair in the demo**

Same recorded answers.
Published as v2.

SOURCE CHANGE / COBOL

```
-   GET-REQUEST
+   FETCH-REQUEST
```

**2 systems**

COBOL + legacy Java

**265 tests**

Backend MVP milestone

Recorded demo result; timing varies by run.

09 — THE CUSTOMER

# Start where legacy logic meets the AI roadmap.

Enterprises with inherited or acquired stacks.

THE BUYING TEAM

## Platform.
## Integration.
## AI engineering.

First use case: read-only access to operational data for agents.

TARGET SECTORS

- Finance
- Insurance
- Energy
- Manufacturing
- Logistics
- Public sector

10 — GO TO MARKET & BUSINESS MODEL

# Land with one team.
# Expand with the business.

01

## Start read-only

Connect one team
to one useful workflow.

02

## Prove the value

Make legacy operations
callable and traceable.

03

## Expand coverage

Add systems across
business units and stacks.

PROPOSED REVENUE MODEL

**SaaS by connected systems + active endpoints**

**Enterprise add-ons**

On-premises · compliance · support

BUILT FOR WHAT COMES NEXT

# Your business logic. Ready for its next life.

Start with one repo and one read-only workflow.



## Explore REPLIC8

replic8.work

THE TEAM Jaiom Rola / Aryan / Toni / Krish